

Macro Evals for Agentic Systems

How to move from thousands of agent events to a small number of actionable behavior patterns.

Source — OpenAI Cookbook · Kwatra, Thieme & Strauss (Slalom)

Date — May 25, 2026

Status — KG Note + Full Essay

Agent Evals

Multi-Agent Systems

Harness Engineering

BERTopic

AgentTrace

Observability

Promptfoo

CONTENTS

01 The Core Problem

Why final answers hide upstream failures in multi-agent systems

02 The Two-Tier Eval Architecture

Lower-level evals vs. macro evals — what each layer does

03 The Four-Label Path

case_type → run_outcome → eval_finding → behavior_pattern

04 Trace Documents

Why document construction is the most critical design decision

05 BERTopic-Style Pattern Discovery

The embed → UMAP → HDBSCAN → label → rank pipeline

06 AgentTrace-Style Backward Diagnosis

The suspect score formula and what it tells you

07 What This Changes About Agent Development

Practical implications for engineering teams and business stakeholders

08 Open Tensions

Where the framework is incomplete or contested

The Core Problem

When an agentic system fails, the failure is almost never where you're looking. You're looking at the final output — the release recommendation, the routed order, the generated response. But the actual failure happened three steps earlier: the pricing agent ignored a promotional signal; the supply specialist missed a stockout; the orchestrator routed around a review step that should have been required.

KEY INSIGHT

"A final answer is only the last event in a longer workflow. A release recommendation can look plausible while the trace reveals that the pricing agent ignored an incentive, the supply agent missed a stockout, or the orchestrator routed around a required review step."

For a simple LLM call, you compare the output against a rubric or reference answer. The pipeline is short enough that inspection is tractable. For a multi-agent system, the workflow may contain dozens of handoffs, hundreds of tool calls, and multiple specialist agents — each with different instructions and different information.

You cannot understand the failure by reading the last line. You need **population-level analysis** — not "what went wrong in this run?" but "what pattern of wrong behavior repeats across hundreds of runs?"

That shift from individual trace inspection to population analysis is the defining move of macro evals.

The Two-Tier Eval Architecture

LOWER-LEVEL EVALS

Grade individual agents, handoffs, tools, and completed runs. Per-trace, per-decision, per-rubric. Produces a local symptom signal for each run.

MACRO EVALS

Look across many lower-level findings. Ask: which problems repeat, where do they concentrate, which part of the workflow should we inspect first?

The Promptfoo Rubrics (Lower-Level)

In the OpenAI cookbook's automotive EV simulation, Promptfoo grades each completed trace against five rubrics:

RUBRIC	QUESTION
<code>final_decision_quality</code>	Does the final decision follow from the active issues and terminal state?
<code>policy_compliance_correctness</code>	Were pricing, tariff, incentive, and compliance constraints handled correctly?
<code>routing_specialist_activation</code>	Did the orchestrator activate the specialists implied by the case?
<code>market_drift_awareness</code>	Did the run respond to dated market signals rather than acting as if the world were static?
<code>review_appropriateness</code>	Was review and escalation proportionate to the case risk?

Each rubric produces a binary signal: pass or fail. A failing lower-level eval is a **local symptom** — one trace, one rubric, one thing that looked wrong. The macro layer discovers what those symptoms become at scale.

THE CORE RELATIONSHIP

Lower-level evals create the signal. Macro evals amplify the signal that repeats. A macro eval without lower-level evals produces meaningless clusters. Lower-level evals without macro analysis miss the patterns invisible in single-run inspection.

The Four-Label Path

Everything in the macro eval framework flows through a single analytical path:



LABEL	WHAT IT ANSWERS	WHEN IT'S KNOWN
case_type	What was the system supposed to do? (e.g. clean order, supplier substitution, pricing exception)	Before the run — generated at scenario setup
run_outcome	How did the run end? (completed, awaiting_review, blocked, failed)	At run completion
eval_finding	What specifically looked wrong or risky? (the Promptfoo rubric that failed)	After lower-level eval grading
behavior_pattern	What recurring cluster do similar failures belong to?	After BERTopic-style discovery — discovered, not defined

The Sankey Diagram — Before and After Clustering

The cookbook uses two Sankey diagrams to show the value of adding `behavior_pattern`:

First Sankey (pre-clustering): `case_type` → `run_outcome` → `eval_finding`. Already useful — shows which scenarios produce which outcomes and which lower-level failures. But cannot answer whether fifty different `eval_finding` signals represent the same underlying workflow behavior.

Second Sankey (post-clustering): adds `behavior_pattern` as the final node. Now the question becomes: did those fifty local symptoms resolve into three or four recurring operating patterns? A business reader can understand what a "fulfillment reroute pattern" means. An AI engineer can use it as a debugging starting point.

WHY THE SANKEY MATTERS

The same artifact communicates to both audiences simultaneously. The four-label path is the translation layer between technical signals and operational meaning.

Trace Documents

The most underrated design decision in the pipeline

Before any clustering can happen, each trace must be converted into a comparable text document. Raw agent traces are not comparable — they're dense with hundreds of events, long model responses, tool payloads, and repeated status updates.

The document construction step compresses each trace into a structured summary that preserves what matters for macro analysis. **What you include in the document determines what the clustering algorithm is allowed to notice.**

What a Good Trace Document Contains

- The business setup — case_type, active environment signals, selected route
- The run outcome and severity
- Important handoffs and specialist activations
- Review and finding markers
- A short state-transition digest

DIRECT QUOTE

"Spend real time on document construction. Better documents usually produce more useful behavior patterns than more complicated clustering settings."

This is a strong and uncomfortable claim. It means that tuning UMAP neighbors and HDBSCAN cluster sizes matters less than getting the document right. It's a data quality argument applied to an intermediate preprocessing step that most teams treat as mechanical.

Focus Events — What to Look For in Traces

When building documents and running diagnosis, anchor on these observability signals:

SIGNAL	MEANING	HOW TO USE
<code>review_finding</code>	A review surface recorded an issue	Start here when the trace has an explicit finding
<code>awaiting_review</code>	The process paused for review	Check if review was justified by actual risk

SIGNAL	MEANING	HOW TO USE
<code>failed / blocked</code>	Degraded terminal state	Walk backward to last handoff, tool, or decision
<code>triage reroute</code>	Workflow changed ownership or path	Inspect whether routing matched case type and signals
<code>tool warning / policy marker</code>	Structured tool exposed risk	Check whether later decisions used or ignored the signal

BERTopic-Style Pattern Discovery

The discovery pass implements a modular four-step pipeline to find recurring behavior patterns across the trace population:

- 1 **Embed** — Each trace t is embedded as $e_t = \text{BERT}(t)$. Traces with similar content end up near each other in the high-dimensional space.
- 2 **Reduce** — UMAP reduces the embedding to a lower-dimensional point z that preserves local neighborhood structure, making clustering tractable.
- 3 **Cluster** — HDBSCAN groups nearby points into dense clusters. Outliers are marked as noise (label -1) — not discarded.
- 4 **Label** — For each cluster, compute a class-aware term score to identify distinctive terms. The top terms become the behavior_pattern label.

Key Formulas

CLASS-AWARE TERM SCORE

$$\text{score}(t, k) = \text{tf}(t, k) \times \log\left(\frac{(1 + N)}{(1 + \text{df}(t))}\right)$$

$\text{tf}(t, k)$ — frequency of term t inside cluster k
 $\text{df}(t)$ — number of clusters where term t appears
 N — total comparison population size

IMPACT SCORE (PATTERN RANKING)

$$\text{impact_score}(k) = \text{prevalence_share}(k) \times \text{severity_weighted_prevalence}(k)$$

LIFT (CONCENTRATION ANALYSIS)

$$\text{lift} = \text{slice_pattern_share} / \text{overall_pattern_share}$$

$\text{lift} > 1.0$ — pattern is concentrated in this scenario slice
 $\text{lift} = 1.0$ — pattern appears at the same rate as overall
 $\text{lift} < 1.0$ — pattern is less common in this slice

Reading the Discovery Output

The topic table is a **triage board**:

COLUMN	WHAT IT TELLS YOU
<code>trace_count</code>	How often the pattern appears — raw frequency
<code>severity_weighted_prevalence</code>	How severe those traces tend to be
<code>impact_score</code>	Combined ranking — where to look first
<code>dominant_owner</code>	Heuristic agent/team label — not an assignment
<code>keywords_text</code>	The terms that made this cluster distinctive

▲ DESIGN TENSIONS

- HDBSCAN and UMAP are sensitive to hyperparameters — different settings produce different taxonomies from the same data. The pattern taxonomy is not stable across runs.
- Rare but severe failures may never reach `min_cluster_size` and will be labeled as noise. These outlier traces may be the most important to investigate.
- Discovery requires a minimum trace population (~100–200+) before clusters become meaningful. Not applicable to brand-new systems.

AgentTrace-Style Backward Diagnosis

Discovery tells you which pattern to investigate. Diagnosis tells you where to look inside that pattern.

The diagnosis pass reconstructs a lightweight execution graph $G = (V, E)$ for traces in the selected pattern, where nodes are normalized events and edges link through temporal order, handoffs, tool calls, and proximity. A **focus event** (anchor) is chosen, and the algorithm walks backward — scoring upstream events as suspects.

The Suspect Score Formula

SUSPECT SCORE

$$\text{suspect_score} = 0.4 \cdot \text{proximity} + 0.3 \cdot \text{frequency} + 0.2 \cdot \text{bridge} + 0.1 \cdot \text{role}$$

COMPONENT	WEIGHT	WHAT IT REWARDS
Proximity	0.4	Events close to the focus event in the execution graph
Frequency	0.3	Events that recur across many traces in the same behavior pattern
Bridge	0.2	Events that connect otherwise separated parts of the graph — structural hubs
Role	0.1	Events whose agent or tool role is plausibly related to the finding type

This is not proof of causality. It is a directed inspection queue — "if I have to choose where to spend debugging time, which upstream events are most likely to be relevant?"

What the Output Tells Each Audience

FOR AI ENGINEERS

Inspect agent instructions and tool contracts for named agents. Check handoff rules around the repeated transition. Determine whether review markers fire too early or too late. Investigate whether tool output is ignored or over-weighted.

FOR BUSINESS STAKEHOLDERS

Identifies which business function appears to own the pattern — fulfillment, pricing, compliance, clarification. The same pattern should bring the corresponding business owners into the next review, not only the prompt engineer.

What This Changes About Agent Development

Evals Must Be Designed, Not Bolted On

The document construction step — what to include in a trace document — is an eval design decision made before any algorithm runs. An eval framework that only grades final output quality will never discover routing failures or market-drift blindness, because those signals aren't in the lower-level labels. Eval design is upstream of discovery.

Population-Scale Infrastructure Is Non-Negotiable

Single-run inspection does not scale. You need: trace collection at runtime, lower-level eval grading per run, document construction per run, batch embedding, clustering, and backward graph analysis. This is a substantial infrastructure investment — and its absence means flying blind.

Synthetic Simulation Is a First-Class Eval Tool

The cookbook runs on 992 synthetic traces from a simulated EV workflow. This is a deliberate design choice, not a limitation. Simulation lets you:

- Build the trace population needed for clustering before you have production data
- Test the eval pipeline against known case types before deployment
- Stress-test specific failure modes by generating more traces in that regime

Next Steps for Each Audience

AI ENGINEERING TEAM

Promote clearest lower-level eval failures into a regression suite. Review automated grades to calibrate rubric strictness. Track behavior patterns by model version, prompt version, and orchestration mode. Inspect top suspect agents and tools before changing the system.

BUSINESS STAKEHOLDER

Decide whether generated case types match real operating risks. Check whether high-impact patterns correspond to important customer outcomes. Validate whether review thresholds produce intended business behavior. Use the heatmap to prioritize scenario improvement.

Open Tensions

▲ WHERE THE FRAMEWORK IS INCOMPLETE

- **Rubric quality is the ceiling** — if the lower-level rubric misses a failure mode, the macro layer never sees it. Garbage in, garbage out at the top of the pipeline.
- **Macro evals are retrospective** — they need a population of completed traces before patterns emerge. They cannot catch a new failure mode on the first run.
- **Document design is subjective** — different document constructions produce different discovered patterns from the same traces. The taxonomy is not objective.
- **Suspect score weights are heuristic** — the 0.4/0.3/0.2/0.1 split is explainable but not empirically calibrated. Different weights would produce a different inspection queue.
- **Backward walk is depth-bounded** — `max_depth=5` means multi-step failures that propagate over many hops get truncated. The actual culprit may be upstream of the diagnostic window.
- **Infrastructure barrier** — the full pipeline (trace collection → lower-level eval → document construction → embedding → clustering → backward diagnosis) is non-trivial to build. There is a maturity gap between well-resourced labs and indie developers.

The Four Levels — Summary

#	Level	What happens	Output
1	Simulation setup	Generate a realistic population of agent runs — diverse scenarios, world conditions, edge cases	Trace bundles (events, spans, env signals, review packets)
2	Lower-level evals	Grade each run against rubrics — decision quality, policy compliance, routing, market awareness, review calibration	eval_finding per trace (pass / fail per rubric)
3	Macro discovery	Embed trace documents → UMAP → HDBSCAN → label clusters → rank by impact → analyze lift by slice	behavior_pattern taxonomy with impact scores and concentration heatmap
4	Trace diagnosis	For highest-impact pattern: reconstruct execution graph → anchor on focus event → backward walk → score suspects	Actionable inspection queue with story strip and swimlane visualization

THE CORE LESSON

"Agent-level evals tell us which local behaviors look risky, while macro evals tell us what those risks become at system scale. The workflow scales by directing human attention toward patterns that are both frequent and consequential — instead of reading every trace, you read the right twenty."